

GAS Data Warehouse

Upcoming Work Items — Post-Handoff Priorities

Prepared as part of Data Warehouse Engineer offboarding · July 2026

This document outlines four outstanding engineering items for the gas_db Data Warehouse and its supporting Azure Data Factory (ADF) / ADLS Gen2 pipelines. Items are listed in priority order (highest to lowest) based on production impact observed to date. Each section describes the issue, the evidence behind it, and recommended remediation options for the team to scope and execute.

1. Bronze Layer Small-File Accumulation (Events Data)

PRIORITY: HIGH

Problem

The CosmosDB Change Feed → Azure Function → Message Bus → ADLS Bronze ingestion path writes one JSON file per individual event/order as it is generated, with no batching or windowing at the consumer level. For events data specifically, this produces an extremely high file count per hourly partition, which the downstream ADF Copy Activity (Bronze → Silver) must enumerate and read on every run.

Evidence

A single 1-hour events folder was profiled in a production Copy Activity run (Activity Run ID 307d0c1a-d81c-4259-8851-b91570cd8e0d, 7/29/2026):

Metric	Events (1-hour folder)	Orders (1-hour folder)
File count	200,000 – 350,000	100 – 2,000
Avg. file size	~2.5 KB	larger, batched
Copy Activity duration	30 – 45 minutes	2 – 3 minutes
Throttling errors observed	18 (single 1-hr run)	none observed
Throughput	~352.5 KB/s	normal

The events pipeline runs 4x/day, meaning each scheduled run accumulates roughly 6–7 hours of backlog — i.e., 1.5–2.5 million tiny files processed in a single Copy Activity execution. The same run reported 18 throttling errors against the ADLS Gen2 source, with throughput of only ~352.5 KB/s despite moving 706 MB of data — confirming that request overhead (file listing + per-file reads), not data volume, is the bottleneck.

Root Cause

- No batching/windowing at the Message Bus → ADLS consumer; each event triggers its own individual blob write.
- Orders data is not materially affected (100–2,000 files/hour, 2–3 min copy duration) — the issue is specific to events/session volume.

Recommended Remediation (in order of impact)

- Batch writes at the Message Bus consumer — buffer events and flush on a time and/or size window (e.g., every 30–60 seconds or every 5,000–10,000 events). This removes the problem at the source; Bronze never receives tiny files. Requires changes to the Azure Function / consumer logic.
- Scheduled compaction sweep on Bronze — a periodic job consolidates accumulated tiny JSON files per hourly partition into consolidated NDJSON files before the ADF Copy Activity runs, then archives/deletes the originals. More containable if changing the Message Bus consumer isn't feasible short term.
- Interim mitigation (no development required) — reduce parallelCopies on the affected Copy Activity (currently 32) to 4–8 to reduce throttling-triggered retries, and/or increase ADF trigger frequency from 4x/day to hourly to shrink the per-run file burst. Neither fixes the root cause, but both reduce failure risk until a proper fix ships.
- Orders data: same batching/compaction approach is a nice-to-have for consistency and future-proofing, but is not currently causing measurable performance or reliability issues.

Urgency

The pipeline is currently succeeding despite throttling, but retries and failure risk will increase as event volume grows. This should be treated as the top-priority item.

2. CI/CD Automation for Schema Migrations & ADF Deployments

PRIORITY: MEDIUM

Problem

CI/CD has not yet been implemented for gas_db schema migrations (DDL, stored procedures) or for ADF pipeline/environment-specific deployments. These have been performed manually to date due to time constraints, which introduces deployment risk (human error, inconsistent environment parity, no audit trail) and slows down future iteration.

Recommended Remediation

- Implement GitHub Actions workflows in the corresponding GitHub repository to automate environment-specific deployment for both the PostgreSQL schema (DDL/stored procedure migrations) and ADF ARM template / pipeline deployments.
- Suggested workflow structure: a build/validation stage (lint SQL, validate ARM templates) followed by environment-gated deployment stages (Dev → Test → Prod), using GitHub Environments with required approvals for Prod.
- ADF deployment should use the standard ARM template export + parameterization approach (linked service connection strings, integration runtime references, etc. as pipeline parameters/secrets) so the same workflow can target multiple environments.
- Schema migrations should be version-controlled and applied idempotently (e.g., a migrations folder with sequentially numbered scripts and a tracking table), rather than manual ad hoc execution against Prod.

Why This Matters

Manual deployment is a single point of failure and knowledge dependency. Automating this removes reliance on any one person's manual steps, creates an auditable deployment history, and reduces the risk of environment drift between Dev, Test, and Prod.

3. ADLS Bronze Tier Optimization (Hot → Cool/Archive)

PRIORITY: MEDIUM–LOW

Problem

The ADLS Bronze layer currently stores all files in Hot tier indefinitely, including files that have already been successfully processed and loaded into the gas_db PostgreSQL Data Warehouse. Since Bronze data is not queried directly once it has been copied into Silver/Gold, retaining it in Hot tier is an unnecessary and avoidable storage cost.

Recommended Remediation

- Once ADF has completed copying a given hourly partition's orders/events JSON files from Bronze into the PostgreSQL Data Warehouse, transition those files to Cool or Archive tier.
- etl.bronze_partition_registry is the natural control point for this: its status column (e.g., completed, not found, pending) already tracks which hourly partitions have been fully processed and can drive a scheduled tiering job — e.g., a policy or Function that queries for status = 'completed' partitions older than a defined threshold and applies a lifecycle/tier change to the corresponding ADLS folder.
- Azure Blob Storage lifecycle management policies (rule-based, based on last-modified date and blob path prefix) could also be considered as a lower-effort alternative to a custom job, if partition completion timing is predictable enough to express as a simple age-based rule.
- Archive tier has rehydration latency/cost — recommend Cool tier unless Bronze retention requirements call for long-term cold storage with rare access.

Why This Matters

This is a cost-optimization item rather than a reliability or correctness issue, which is why it is ranked last — but the volume of files involved (millions per day, per Issue #1) means the potential savings from moving processed data out of Hot tier are likely meaningful and worth scoping.

4. ML Schema Transaction Table Cleanup (Post-Extraction)

PRIORITY: MEDIUM–LOW

Problem

The ml schema's order/transaction-oriented tables (used for fuzzy-dedup and feature preparation ahead of model training) grow at roughly the same rate as the corresponding fact tables, since they carry comparable transaction-level granularity. Left unmanaged, this data accumulates indefinitely even after it has already served its purpose of being extracted for training, adding unnecessary storage and maintenance overhead (indexing, vacuum, backup size) with no ongoing analytical value.

Recommended Remediation — Two Options

- Option A — Organization/location-scoped deletion: once training data for a specific organization or location has been extracted and dumped into the SUS Data Lake, delete the corresponding rows from the ml schema transaction tables for that org/location. This aligns cleanup directly with the existing per-org/per-location extraction trigger, keeping only what hasn't yet been extracted.
- Option B — Bulk extract-then-wipe: instead of gating cleanup on a per-org/per-location ML extraction trigger, proactively extract transaction data for all organizations into SUS ADLS on a scheduled basis, then wipe the ml

schema transaction tables in bulk once extraction is confirmed complete. Simpler to schedule and reason about, but requires confidence that all downstream ML consumers are satisfied by the bulk extraction cadence before deletion.

- Either option should reuse the existing p_organizationid-scoped incremental refresh pattern already built into the ml schema stored procedures, so deletion logic stays consistent with how the tables are populated.
- Recommend confirming with the ML/Data Science team which extraction pattern (per-org/location trigger vs. scheduled bulk) better matches their actual consumption pattern before committing to Option A or B.

Why This Matters

This is ranked below the other items because it does not currently cause failures or measurable performance degradation, but the growth rate mirrors fact-table volume, so the storage and maintenance cost will continue compounding if left unaddressed.

Summary

#	Task	Priority	Est. Effort
1	Bronze small-file remediation (events priority; orders nice-to-have)	HIGH	Medium–High
2	CI/CD automation for schema migrations & ADF deployment (GitHub Actions)	MEDIUM	Medium
3	ADLS Bronze tiering (Hot → Cool/Archive) after processing	MEDIUM	Low–Medium
4	ML schema transaction table cleanup post-extraction	MEDIUM–LOW	Medium